ORBIT · PYTHON · PAPER 01

第 1 套

60 题 · 完整题目与答案

单选 30 多选 15 填空 / 简答 15

本套分节

一、单项选择题（30） 二、不定项选择题（15） 三、填空题（5） 四、程序代码阅读题（5） 五、程序设计题（5）

答案与解析紧随每题；本文件用于复习核对，建议先独立作答。

一、单项选择题

01 • 单项选择

在Python中，以下哪个方法用于创建类的实例？

A. `__new__()`
B. `__init__()`
C. `__create__()`
D. `__instance__()`

答案 B. `__init__()`

解析 Python中创建实例涉及两个方法：
• `__new__()` ：真正的构造方法，负责创建对象实例，返回实例对象
• `__init__()` ：初始化方法，在对象创建后调用，用于初始化对象属性
执行顺序： `__new__()` → `__init__()`
示例：

```
class MyClass:
    def __new__(cls):
        print("调用__new__，创建实例")
        return super().__new__(cls)
    def __init__(self):
        print("调用__init__，初始化实例")
```

知识拓展：通常我们说的"构造方法"指 `__init__()` ，因为 `__new__()` 很少需要重写。 `__new__()` 的主要应用场景是实现单例模式或继承不可变类型（如int、str）。

02 • 单项选择

关于Python的 `self` 参数，以下说法正确的是：

A. `self` 是Python的关键字
B. `self` 必须作为实例方法的第一个参数
C. `self` 可以用任何名称替代
D. B和C都正确

答案 D. B和C都正确

解析 • `self` 不是Python关键字，只是一个约定俗成的命名
• 可以用其他名称（如 `this` 、 `me` ）替代，但不推荐
• 实例方法必须将实例本身作为第一个参数
示例：

```
class Test:
    def method1(self):
        # 标准用法
        print(self)
    def method2(this):
        # 可以，但不推荐
        print(this)
```

知识拓展：类方法使用 `cls` 作为第一个参数，静态方法不需要特殊的第一个参数。

03 • 单项选择

以下哪个装饰器用于定义静态方法？

A. `@staticmethod`
B. `@classmethod`

C. @property

D. @static

答案 A. @staticmethod

解析 Python提供三种方法装饰器：

- @staticmethod ：静态方法，不需要访问类或实例
- @classmethod ：类方法，第一个参数是类本身（cls）
- @property ：属性装饰器，将方法转为属性访问

对比示例：

```
class MyClass:
    @staticmethod
    def static_method():
        return "静态方法"
    @classmethod
    def class_method(cls):
        return f"类方法，类名：{cls.__name__}"
    @property
    def my_property(self):
        return "属性"
```

04 • 单项选择

Python中， __str__() 和 __repr__() 的主要区别是：

A. 完全相同

B. __str__() 面向用户， __repr__() 面向开发者

C. __str__() 面向开发者， __repr__() 面向用户

D. 两者都不能被重写

答案 B. __str__() 面向用户， __repr__() 面向开发者

解析 • __str__() ：返回用户友好的字符串，用于 print() 和 str()

- __repr__() ：返回开发者友好的字符串，用于调试和 repr()
- 理想情况下， eval(repr(obj)) == obj

示例：

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y
    def __str__(self):
        return f"点({self.x}, {self.y})"
    # 用户友好
    def __repr__(self):
        return f"Point({self.x}, {self.y})"
# 开发者友好
p = Point(3, 4)
print(str(p))
# 点(3, 4)
print(repr(p))
# Point(3, 4)
```

05 • 单项选择

以下代码的输出是什么？

```
class Counter:
    count = 0
    def __init__(self):
        Counter.count += 1
c1 = Counter()
c2 = Counter()
print(Counter.count)
```

A. 0
B. 1
C. 2
D. 报错

答案 C. 2

解析 这是类变量（类属性）的使用示例：
- count 是类变量，所有实例共享
- 每次创建实例时， __init__() 中 Counter.count += 1 执行一次
- 创建c1时：count = 0 + 1 = 1
- 创建c2时：count = 1 + 1 = 2

类变量vs实例变量：

```
class Demo:
    class_var = 0
    # 类变量，所有实例共享
    def __init__(self):
        self.instance_var = 0
        # 实例变量，每个实例独有
        Demo.class_var += 1
```

06 • 单项选择

关于Python的继承，以下说法错误的是：

A. Python支持多重继承
B. 子类会继承父类的所有属性和方法
C. 子类必须重写父类的所有方法
D. 使用 super() 可以调用父类方法

答案 C. 子类必须重写父类的所有方法

解析 Python继承的特点：
- 支持单继承和多重继承
- 子类自动继承父类的所有属性和方法
- 子类可以选择性
    可以选择性重写父类方法（方法覆盖）
- 使用 super() 调用父类方法

示例：

```
class Parent:
    def method1(self):
        print("父类方法1")
    def method2(self):
        print("父类方法2")
class Child(Parent):
```

```
    def method1(self):
        # 重写method1
        print("子类方法1")
# method2不重写，直接继承
```

07 • 单项选择

列表推导式 `[i**2 for i in range(6) if i % 2 == 1]` 的结果是：

A. [1, 9, 25]
B. [0, 4, 16]
C. [1, 3, 5]
D. [0, 1, 4, 9, 16, 25]

答案 A. [1, 9, 25]

解析 列表推导式的执行流程：
`[i**2 for i in range(6) if i % 2 == 1]`
逐步分析：
- range(6) 生成：0, 1, 2, 3, 4, 5
- if i % 2 == 1 过滤奇数：1, 3, 5
- i**2 计算平方：1^2=1, 3^2=9, 5^2=25
- 结果： [1, 9, 25]

08 • 单项选择

以下哪个不是可迭代对象？

A. 列表
B. 字符串
C. 整数
D. 字典

答案 C. 整数

解析 可迭代对象必须实现 __iter__() 方法或 __getitem__() 方法：
- 列表、元组、字符串、字典、集合：都是可迭代对象
- 整数、浮点数：不是可迭代对象

验证方法：

```
from collections.abc import Iterable
print(isinstance([1, 2], Iterable))
# True
print(isinstance("abc", Iterable))
# True
print(isinstance(42, Iterable))
# False
```

09 • 单项选择

关于生成器，以下说法正确的是：

A. 生成器会一次性生成所有值
B. 生成器使用 yield 关键字
C. 生成器比列表占用更多内存
D. 生成器不能用于循环

答案 B. 生成器使用 yield 关键字

解析 生成器的特点：

- 使用 yield 关键字定义
- 惰性求值

  惰性求值：按需生成值，不会一次性生成所有值
- 内存友好

  内存友好：只保存当前状态，不存储所有值
- 只能迭代一次

生成器vs列表：

```
        列表：
# 列表：立即生成所有值
list_comp = [x**2 for x in range(10000)]
# 占用大量内存
# 生成器：按需生成
gen_exp = (x**2 for x in range(10000))
# 内存占用小
```

10 • 单项选择

以下代码的输出是什么？

```
def gen():
    for i in range(3):
        yield i
g = gen()
print(next(g))
```

A. 0 1

B. 1 2

C. 0 0

D. 报错

答案 A. 0 1

解析 生成器的执行流程：

```
def gen():
    for i in range(3):
        # i = 0, 1, 2
        yield i
```

执行过程：

1. g = gen() ：创建生成器对象，不执行函数体
2. next(g) ：执行到第一个 yield ，返回0
3. next(g) ：继续执行到下一个 yield ，返回1

11 • 单项选择

关于迭代器协议，以下说法正确的是：

A. 只需实现 __iter__() 方法

B. 只需实现 __next__() 方法

C. 需要同时实现 __iter__() 和 __next__() 方法

D. 不需要实现任何特殊方法

答案 C. 需要同时实现 __iter__() 和 __next__() 方法

解析 迭代器协议要求实现两个方法：

- __iter__() ：返回迭代器对象本身

- `__next__()`：返回下一个值，无值时抛出 StopIteration

自定义迭代器：

```
class Counter:
    def __init__(self, max_num):
        self.max_num = max_num
        self.num = 0
    def __iter__(self):
        return self
    def __next__(self):
        if self.num < self.max_num:
            self.num += 1
            return self.num
        raise StopIteration
```

12 • 单项选择

装饰器本质上是：

A. 一个类
B. 一个函数
C. 一个模块
D. 一个关键字

答案 B. 一个函数

解析 装饰器是一个高阶函数：

- 接收一个函数作为参数
- 返回一个新函数
- 本质是函数的包装器

基本结构：

```
def decorator(func):
    def wrapper(*args, **kwargs):
        # 前置处理
        result = func(*args, **kwargs)
        # 后置处理
        return result
    return wrapper
@decorator
def my_function():
    pass
# 等价于：my_function = decorator(my_function)
```

13 • 单项选择

以下代码的输出是什么？

```
def decorator(func):
    def wrapper():
        return func() * 2
    return wrapper
@decorator
def get_num():
    return 5
print(get_num())
```

A. 5
B. 10
C. 25
D. 报错

答案 B. 10

解析 装饰器的执行流程：

```
def double(func):
    def wrapper():
        return func() * 2
    return wrapper
@double
def get_num():
    return 5
```

执行过程：

1. get_num() 被 double 装饰
2. 调用 get_num() 实际调用 wrapper()
3. wrapper() 执行： func() * 2 = 5 * 2 = 10

14 • 单项选择

关于 *args 和 **kwargs ，以下说法正确的是：

A. *args 用于接收位置参数， **kwargs 用于接收关键字参数
B. *args 用于接收关键字参数， **kwargs 用于接收位置参数
C. 两者完全相同
D. 只能使用这两个名称

答案 A. *args 用于接收位置参数， **kwargs 用于接收关键字参数

解析 Python的可变参数：

- *args ：接收任意数量的位置参数，存为元组
- **kwargs ：接收任意数量的关键字参数，存为字典

示例：

```
def func(*args, **kwargs):
    print(f"args: {args}, type: {type(args)}")
    print(f"kwargs: {kwargs}, type: {type(kwargs)}")
func(1, 2, 3, a=4, b=5)
# args: (1, 2, 3), type: <class 'tuple'>
# kwargs: {'a': 4, 'b': 5}, type: <class 'dict'>
```

15 • 单项选择

以下代码的输出是什么？

```
a, b, *c = [1, 2, 3, 4, 5]
print(c)
```

A. [3, 4, 5]
B. [1, 2]
C. 3
D. [1, 2, 3, 4, 5]

答案 A. [3, 4, 5]

解析 解包赋值（序列解包）：

```
a, b, *c = [1, 2, 3, 4, 5]
```

执行过程：
- a = 1 （第一个元素）
- b = 2 （第二个元素）
- *c 接收剩余元素： c = [3, 4, 5]

其他用法：

```
a, *b, c = [1, 2, 3, 4, 5]
# a=1, b=[2,3,4], c=5
*a, b, c = [1, 2, 3, 4, 5]
# a=[1,2,3], b=4, c=5
```

16 • 单项选择

在Pandas中，DataFrame是：

A. 一维数据结构
B. 二维数据结构
C. 三维数据结构
D. 零维数据结构

答案 B. 二维数据结构

解析 Pandas核心数据结构：
- Series：一维标签数组，类似带索引的列表
- DataFrame：二维表格数据，类似Excel表格
- Panel：三维数据（已弃用）

DataFrame特点：

特点：
- 行索引（index）
- 列索引（columns）
- 可包含不同数据类型的列

17 • 单项选择

Pandas中，以下哪个方法用于读取CSV文件？

A. pd.read_csv()
B. pd.load_csv()
C. pd.import_csv()
D. pd.open_csv()

答案 A. pd.read_csv()

解析 Pandas常用读取函数：
- pd.read_csv() ：读取CSV文件
- pd.read_excel() ：读取Excel文件
- pd.read_json() ：读取JSON文件
- pd.read_sql() ：从数据库读取

18 • 单项选择

关于Pandas的 loc 和 iloc ，以下说法正确的是：

A. loc 使用整数位置索引
B. iloc 使用标签索引
C. loc 使用标签索引， iloc 使用整数位置索引
D. 两者完全相同

答案 C. loc 使用标签索引， iloc 使用整数位置索引

解析 Pandas索引方法对比：

- loc ：基于标签
  标签的索引，使用行列名称
- iloc ：基于整数位置
  整数位置的索引，使用数字下标

示例：

```
df = pd.DataFrame({'A': [1,2,3]}, index=['x','y','z'])
df.loc['x', 'A']
# 使用标签
df.iloc[0, 0]
# 使用位置
```

19 • 单项选择

以下哪个Pandas方法用于删除缺失值？

A. dropna()
B. fillna()
C. removena()
D. deletena()

答案 A. dropna()

解析 Pandas处理缺失值的方法：

- dropna() ：删除包含缺失值的行/列
- fillna() ：填充缺失值
- isna() / isnull() ：检测缺失值

20 • 单项选择

Pandas中， groupby() 操作返回的是：

A. DataFrame
B. Series
C. GroupBy对象
D. 列表

答案 C. GroupBy对象

解析 groupby() 返回GroupBy对象，需要进一步操作：

```
df.groupby('column').mean()
# 计算均值
df.groupby('column').sum()
# 计算总和
df.groupby('column').agg({})
# 聚合操作
```

21-25. Path对象属性：

21 • 单项选择

使用 pathlib.Path ，以下哪个属性返回文件名（不含路径）？

A. path.filename
B. path.name
C. path.basename
D. path.file

答案 B. path.name

解析

22 • 单项选择

Path 对象的 suffix 属性返回的是：

A. 文件路径
B. 文件扩展名
C. 文件名
D. 父目录

答案 B. 文件扩展名

解析

23 • 单项选择

以下哪个方法用于检查路径是否存在？

A. path.exists()
B. path.check()
C. path.is_exist()
D. path.valid()

答案 A. path.exists()

解析

24 • 单项选择

关于 Path 对象的 / 运算符，以下说法正确的是：

A. 用于除法运算
B. 用于路径拼接
C. 用于路径分割
D. 不能使用

答案 B. 用于路径拼接

解析

25 • 单项选择

以下代码的输出是什么？

```
from pathlib import Path
p = Path('/home/user/file.txt')
print(p.stem)
```

A. file.txt
B. file
C. .txt
D. /home/user

答案 B. file

解析 Path对象示例：

```
from pathlib import Path
p = Path('/home/user/document.txt')
print(p.name)
# 'document.txt'
```

```
print(p.suffix)
# '.txt'
print(p.stem)
# 'document'
print(p.parent)
# '/home/user'
```

26 • 单项选择

关于Python的 lambda 函数，以下说法正确的是：

A. 可以包含多条语句
B. 只能包含一个表达式
C. 必须有函数名
D. 不能作为参数传递

答案 B. 只能包含一个表达式

解析 Lambda函数的限制：
• 只能包含一个表达式
• 不能包含语句（如赋值、循环）
• 自动返回表达式的结果
对比：

```
# lambda：单一表达式
lambda x: x * 2
# 普通函数：可包含多条语句
def func(x):
    result = x * 2
    return result
```

27 • 单项选择

以下代码的输出是什么？

```
nums = [1, 2, 3, 4]
result = list(map(lambda x: x * 2, nums))
print(result)
```

A. [1, 2, 3, 4]
B. [2, 4, 6, 8]
C. [1, 4, 9, 16]
D. 报错

答案 B. [2, 4, 6, 8]

解析 map() 函数将函数应用于序列的每个元素：

```
list(map(lambda x: x * 2, [1, 2, 3]))
# 结果：[2, 4, 6]
```

28 • 单项选择

字典推导式 {k: v**2 for k, v in {'a': 1, 'b': 2}.items()} 的结果是：

A. {'a': 1, 'b': 2}
B. {'a': 1, 'b': 4}
C. {'a': 2, 'b': 4}
D. 报错

答案 B. {'a': 1, 'b': 4}

解析 字典推导式：

```
{k: v**2 for k, v in {'a': 2, 'b': 3}.items()}
# 结果：{'a': 4, 'b': 9}
```

29 • 单项选择

关于 zip() 函数，以下说法错误的是：

A. 返回一个迭代器
B. 可以同时迭代多个序列
C. 以最长的序列为准
D. 返回元组的迭代器

答案 C. 以最长的序列为准

解析 zip() 以最短序列
最短序列为准：

```
list(zip([1,2,3], ['a','b']))
                # [(1,'a'), (2,'b')]
# 第三个元素3被丢弃
```

30 • 单项选择

以下代码的输出是什么？

```
list1 = [1, 2, 3]
list2 = ['a', 'b', 'c']
result = list(zip(list1, list2))
print(len(result))
```

A. 2
B. 3
C. 6
D. 报错

答案 B. 3

解析 两个长度为3的列表zip后，结果长度为3：

```
list(zip([1,2,3], [4,5,6]))
# [(1,4), (2,5), (3,6)]
```

二、不定项选择题

31 • 不定项选择

关于Python类的特殊方法，以下说法正确的有：

A. __init__() 是构造方法
B. __del__() 是析构方法
C. __len__() 用于返回对象长度
D. __call__() 使对象可以像函数一样被调用

答案 A. __init__() 是构造方法；B. __del__() 是析构方法；C. __len__() 用于返回对象长度；D. __call__() 使对象可以像函数一样被调用

解析 @property 装饰器的作用：

- A. 将方法转换为属性

将方法转换为属性　正确，让方法像属性一样访问

- B. 可以实现

可以实现getter和setter　正确，配合 @property.setter 使用

- C. 提供只读属性

提供只读属性　正确，不定义setter时即为只读

- D. 可以添加数据验证逻辑

可以添加数据验证逻辑　正确，在setter中可以验证

完整示例：

```
class Temperature:
    def __init__(self, celsius):
        self._celsius = celsius
    @property
    def celsius(self):
        """Getter: 获取温度"""
        return self._celsius
    @celsius.setter
    def celsius(self, value):
        """Setter: 设置温度并验证"""
        if value < -273.15:
            raise ValueError("温度不能低于绝对零度")
        self._celsius = value
    @property
    def fahrenheit(self):
        """只读属性：摄氏转华氏"""
        return self._celsius * 9/5 + 32
```

32 • 不定项选择

以下哪些是Python的访问控制约定：

A. 单下划线开头表示受保护的成员

B. 双下划线开头表示私有成员

C. 双下划线开头和结尾表示特殊方法

D. Python有严格的访问控制机制

答案 A. 单下划线开头表示受保护的成员；B. 双下划线开头表示私有成员；C. 双下划线开头和结尾表示特殊方法

解析 Python访问控制约定：

- A. 单下划线

单下划线 _ 表示保护属性

表示保护属性　约定，不强制

- B. 双下划线

双下划线 __ 触发名称改变

触发名称改变　正确，但不是真正的私有

- C. 没有严格的访问控制

没有严格的访问控制　Python信任程序员

- D. 可以实现真正的私有属性

可以实现真正的私有属性　错误，Python无法完全隐藏

名称改变机制：

```
class MyClass:
    def __init__(self):
        self._protected = 1
```

```
        # 保护属性（约定）
        self.__private = 2
# 私有属性（名称改变）
obj = MyClass()
print(obj._protected)
# 1 - 可以访问
print(obj._MyClass__private)
# 2 - 仍然可以访问!
```

33 • 不定项选择

关于Python的多重继承，以下说法正确的有：

A. Python支持多重继承
B. 使用MRO（方法解析顺序）确定方法调用顺序
C. 可以使用 __mro__ 属性查看继承顺序
D. 多重继承可能导致菱形继承问题

答案 A. Python支持多重继承；B. 使用MRO（方法解析顺序）确定方法调用顺序；C. 可以使用 __mro__ 属性查看继承顺序；D. 多重继承可能导致菱形继承问题

解析 生成器的优点：

- A. 节省内存
  节省内存　不一次性生成所有值
- B. 惰性求值
  惰性求值　按需计算
- C. 适合处理大数据集
  适合处理大数据集　内存占用小
- D. 可以表示无限序列
  可以表示无限序列　不需要预先生成所有值

对比示例：

```
import sys
# 列表：立即生成所有值
list_data = [x**2 for x in range(1000000)]
print(f"列表内存：{sys.getsizeof(list_data)} 字节")
# 生成器：按需生成
gen_data = (x**2 for x in range(1000000))
print(f"生成器内存：{sys.getsizeof(gen_data)} 字节")
# 无限序列示例
def infinite_sequence():
    num = 0
    while True:
        yield num
        num += 1
```

34 • 不定项选择

以下哪些是生成器的创建方式：

A. 使用 yield 关键字的函数
B. 使用生成器表达式 ()
C. 使用列表推导式 []
D. 使用 iter() 函数

答案 A. 使用 yield 关键字的函数；B. 使用生成器表达式 ()

解析 生成器的创建方式：

- A. 使用
  使用 yield 的函数
  的函数 生成器函数
- B. 生成器表达式
  生成器表达式 (x for x in ...)
- C. 列表推导式
  列表推导式 生成列表，不是生成器
- D. 实现
  实现 __iter__() 和 __next__() 这是迭代器，不是生成器

对比：

```
# A. 生成器函数
def gen_func():
    yield 1
    yield 2
# B. 生成器表达式
gen_exp = (x for x in range(3))
# C. 列表推导式（不是生成器）
list_comp = [x for x in range(3)]
```

35 • 不定项选择

关于迭代器和可迭代对象，以下说法正确的有：

A. 所有迭代器都是可迭代对象
B. 所有可迭代对象都是迭代器
C. 迭代器实现了 __iter__() 和 __next__() 方法
D. 可迭代对象只需实现 __iter__() 方法

答案 A. 所有迭代器都是可迭代对象；C. 迭代器实现了 __iter__() 和 __next__() 方法；D. 可迭代对象只需实现 __iter__() 方法

解析 • A. 可迭代对象可以用于
可迭代对象可以用于for循环
循环 正确

- B. 所有可迭代对象都是迭代器
  所有可迭代对象都是迭代器 错误，列表是可迭代但不是迭代器
- C. 迭代器必然是可迭代对象
  迭代器必然是可迭代对象 正确
- D. 可迭代对象实现
  可迭代对象实现 __iter__() 正确

区别：

```
# 可迭代对象：实现__iter__()
my_list = [1, 2, 3]
# 可迭代，但不是迭代器
# 迭代器：实现__iter__()和__next__()
my_iter = iter(my_list)
# 迭代器
```

36 • 不定项选择

以下哪些是装饰器的应用场景：

A. 日志记录
B. 性能测试

C. 权限验证

D. 缓存

答案 A. 日志记录；B. 性能测试；C. 权限验证；D. 缓存

解析 装饰器的常见应用：

• A. 计时统计

计时统计　测量函数执行时间

• B. 权限检查

权限检查　验证用户权限

• C. 缓存结果

缓存结果　避免重复计算

• D. 日志记录

日志记录　记录函数调用

应用示例：

```
import time
from functools import wraps
# 计时装饰器
def timer(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        print(f"{func.__name__} 耗时: {time.time()-start:.4f}秒")
        return result
    return wrapper
# 缓存装饰器
def cache(func):
    cached_results = {}
    @wraps(func)
    def wrapper(*args):
        if args not in cached_results:
            cached_results[args] = func(*args)
        return cached_results[args]
    return wrapper
```

7-15. 答案：

答案：ABCD

这些题目所有选项都正确，分别涉及Python高级特性、Pandas操作、pathlib模块和函数式编程的各个方面。

14. 特别说明：

特别说明：ABC

• D. 列表推导式总是比

列表推导式总是比for循环快

循环快　不一定，复杂逻辑时可能更慢

37 • 不定项选择

关于函数参数，以下说法正确的有：

A. 位置参数必须在关键字参数之前

B. *args 必须在 **kwargs 之前

C. 默认参数必须在位置参数之后

D. 可以使用 * 强制后面的参数必须使用关键字传递

答案 A. 位置参数必须在关键字参数之前；B. *args 必须在 **kwargs 之前；C. 默认参数必须在位置参数之后；D. 可以使用 * 强制后面的参数必须使用关键字传递

解析 （原卷合并作答）

38 • 不定项选择

关于解包操作，以下说法正确的有：

A. * 用于解包序列
B. ** 用于解包字典
C. 可以在函数调用时使用解包
D. 可以在赋值时使用解包

答案 A. * 用于解包序列；B. ** 用于解包字典；C. 可以在函数调用时使用解包；D. 可以在赋值时使用解包

解析 （原卷合并作答）

39 • 不定项选择

Pandas中，以下哪些方法可以用于数据选择：

A. loc[]
B. iloc[]
C. at[]
D. iat[]

答案 A. loc[]；B. iloc[]；C. at[]；D. iat[]

解析 （原卷合并作答）

40 • 不定项选择

关于Pandas的缺失值处理，以下说法正确的有：

A. dropna() 删除缺失值
B. fillna() 填充缺失值
C. isna() 检测缺失值
D. notna() 检测非缺失值

答案 A. dropna() 删除缺失值；B. fillna() 填充缺失值；C. isna() 检测缺失值；D. notna() 检测非缺失值

解析 （原卷合并作答）

41 • 不定项选择

Pandas中，以下哪些方法用于数据合并：

A. merge()
B. concat()
C. join()
D. append()

答案 A. merge()；B. concat()；C. join()；D. append()

解析 （原卷合并作答）

42 • 不定项选择

关于 pathlib.Path ，以下说法正确的有：

A. 提供面向对象的路径操作

B. 跨平台兼容
C. 可以使用 / 运算符拼接路径
D. 比 os.path 更现代

答案 A. 提供面向对象的路径操作；B. 跨平台兼容；C. 可以使用 / 运算符拼接路径；D. 比 os.path 更现代

解析 （原卷合并作答）

43 • 不定项选择

Path 对象的以下哪些方法用于路径判断：

A. is_file()
B. is_dir()
C. exists()
D. is_absolute()

答案 A. is_file()；B. is_dir()；C. exists()；D. is_absolute()

解析 （原卷合并作答）

44 • 不定项选择

关于列表推导式，以下说法正确的有：

A. 可以包含条件判断
B. 可以嵌套使用
C. 比for循环更简洁
D. 总是比for循环快

答案 A. 可以包含条件判断；B. 可以嵌套使用；C. 比for循环更简洁；D. 总是比for循环快

解析 （原卷合并作答）

45 • 不定项选择

以下哪些是Python的高阶函数：

A. map()
B. filter()
C. reduce()
D. sorted()

答案 A. map()；B. filter()；C. reduce()；D. sorted()

解析 （原卷合并作答）

三、填空题

46 • 填空 / 简答

在Python类中，使用 _____ 装饰器可以定义类方法，使用 _____ 装饰器可以定义静态方法。

答案 @classmethod ， @staticmethod

解析 Python中定义类方法和静态方法的装饰器：
类方法 @classmethod ：
- 第一个参数是类本身（约定为 cls ）
- 可以访问类属性和类方法

- 常用于工厂方法、替代构造函数

静态方法 @staticmethod ：

- 不需要 self 或 cls 参数
- 无法访问实例或类属性
- 适合工具函数，逻辑上属于类

对比示例：

```
class Date:
    def __init__(self, year, month, day):
        self.year = year
        self.month = month
        self.day = day
    @classmethod
    def from_string(cls, date_str):
        """类方法：从字符串创建对象"""
        year, month, day = map(int, date_str.split('-'))
        return cls(year, month, day)
    @staticmethod
    def is_valid_date(year, month, day):
        """静态方法：验证日期"""
        return 1 <= month <= 12 and 1 <= day <= 31
# 使用
d1 = Date.from_string('2024-01-15')
# 类方法
valid = Date.is_valid_date(2024, 1, 15)
# 静态方法
```

47 • 填空 / 简答

生成器函数使用 _____ 关键字返回值，每次调用 _____ 函数可以获取生成器的下一个值。

答案 yield ， next()

解析 生成器的核心关键字和函数：

yield 关键字：

关键字：

- 用于定义生成器函数
- 暂停函数执行并返回值
- 保存函数状态，下次调用从暂停处继续

next() 函数：

函数：

- 获取生成器的下一个值
- 无值时抛出 StopIteration 异常

完整示例：

```
def fibonacci(n):
    """生成前n个斐波那契数"""
    a, b = 0, 1
    count = 0
    while count < n:
        yield a
        # 返回当前值并暂停
        a, b = b, a + b
        count += 1
```

```
# 使用next()
fib = fibonacci(5)
print(next(fib))
# 0
print(next(fib))
# 1
print(next(fib))
# 1
# 使用for循环
for num in fibonacci(5):
    print(num, end=' ')
# 0 1 1 2 3
```

48 • 填空 / 简答

在Pandas中，_____ 用于基于标签的索引，_____ 用于基于整数位置的索引。

答案 loc ， iloc

解析 Pandas两种主要的索引方式：

loc - 基于标签的索引：

- 使用行名和列名
- 包含结束位置（闭区间）
- 支持布尔索引

iloc - 基于整数位置的索引：

- 使用整数位置（0开始）
- 不包含结束位置（左闭右开）
- 类似Python列表切片

对比示例：

```
import pandas as pd
df = pd.DataFrame({
    'A': [1, 2, 3],
    'B': [4, 5, 6]
}, index=['x', 'y', 'z'])
# loc: 使用标签
df.loc['x', 'A']
# 1
df.loc['x':'y', 'A']
# 包含'y'
# iloc: 使用位置
df.iloc[0, 0]
# 1
df.iloc[0:2, 0]
# 不包含位置2
# 布尔索引（只能loc）
df.loc[df['A'] > 1, 'B']
# 条件筛选
```

49 • 填空 / 简答

使用 _____ 符号可以解包序列，使用 _____ 符号可以解包字典。

答案 * ， **

解析 Python参数解包和打包符号：
* - 位置参数：
• 函数定义中： *args 打包为元组
• 函数调用中：解包序列为位置参数
• 赋值中：收集剩余元素
** - 关键字参数：
• 函数定义中： **kwargs 打包为字典
• 函数调用中：解包字典为关键字参数
应用场景：

```
# 1. 函数定义：接收任意参数
def func(*args, **kwargs):
    print(f"args: {args}")
    # 元组
    print(f"kwargs: {kwargs}")
# 字典
func(1, 2, 3, a=4, b=5)
# args: (1, 2, 3)
# kwargs: {'a': 4, 'b': 5}
# 2. 函数调用：解包参数
values = [1, 2, 3]
params = {'x': 10, 'y': 20}
def add(a, b, c, x=0, y=0):
    return a + b + c + x + y
result = add(*values, **params)
# 解包
# 3. 赋值中：收集元素
a, *b, c = [1, 2, 3, 4, 5]
# a=1, b=[2,3,4], c=5
```

50 • 填空 / 简答

pathlib.Path 对象的 _____ 属性返回文件扩展名，_____ 属性返回不含扩展名的文件名。

答案 suffix ， stem

解析 羿析： pathlib.Path 对象的文件名相关属性：
suffix - 文件扩展名：
• 返回包括点号的扩展名（如 .txt ）
• 空字符串表示无扩展名
• suffixes 属性返回所有扩展名列表
stem - 不含扩展名的文件名：
• 只返回文件主名部分
• 不包含路径和扩展名
完整对比：

```
from pathlib import Path
# 单个扩展名
p1 = Path('/home/user/document.txt')
print(p1.name)
# 'document.txt' - 完整文件名
print(p1.suffix)
# '.txt' - 扩展名
print(p1.stem)
```

```
# 'document' - 不含扩展名
# 多个扩展名
p2 = Path('/home/user/archive.tar.gz')
print(p2.suffix)
# '.gz' - 最后一个扩展名
print(p2.suffixes)
# ['.tar', '.gz'] - 所有扩展名
print(p2.stem)
# 'archive.tar' - 不含最后扩展名
# 无扩展名
p3 = Path('/home/user/README')
print(p3.suffix)
# '' - 空字符串
print(p3.stem)
# 'README'
# 应用：修改扩展名
old_path = Path('document.txt')
new_path = old_path.with_suffix('.md')
print(new_path)
                # 'document.md'
```

四、程序代码阅读题

51 • 填空 / 简答

阅读以下代码，回答问题：

```
class BankAccount:
    def __init__(self, balance=0):
        self._balance = balance
    @property
    def balance(self):
        return self._balance
    @balance.setter
    def balance(self, value):
        if value < 0:
            raise ValueError("余额不能为负数")
        self._balance = value
    def deposit(self, amount):
        self.balance += amount
account = BankAccount(100)
account.deposit(50)
print(account.balance)
```

问题：

- (1) 输出结果是什么？
- (2) @property 装饰器的作用是什么？
- (3) 如果执行 account.balance = -10 会发生什么？

答案 • (1) 输出：150
• (2) @property 将方法转换为属性，可以像访问属性一样调用方法，同时可以添加getter和

setter进行控制
• (3) 会抛出 ValueError 异常，提示"余额不能为负数"

解析

52 • 填空 / 简答

阅读以下代码，回答问题：

```
def fibonacci_gen(n):
    a, b = 0, 1
    count = 0
    while count < n:
        yield a
        a, b = b, a + b
        count += 1
fib = fibonacci_gen(5)
print(list(fib))
```

问题：
• (1) 第一个 print 输出什么？
• (2) 第二个 print 输出什么？为什么？
• (3) 生成器相比列表的优势是什么？

答案 • (1) 输出： [0, 1, 1, 2, 3]
• (2) 输出： [] 。因为生成器只能迭代一次，第一次迭代后已经耗尽
• (3) 生成器按需生成值，不会一次性占用大量内存，适合处理大数据集

解析

53 • 填空 / 简答

阅读以下代码，回答问题：

```
import pandas as pd
df = pd.DataFrame({
    'name': ['Alice', 'Bob', 'Charlie', 'David'],
    'age': [25, 30, 35, 40],
    'salary': [5000, 6000, 7000, 8000]
})
result = df[df['age'] > 30]['salary'].mean()
print(result)
```

问题：
• (1) 输出结果是什么？
• (2) df['age'] > 30 返回的是什么类型？
• (3) 如何使用 loc 改写这个查询？

答案 数据筛选
• (1) 输出：7500.0（35岁Charlie的7000和40岁David的8000的平均值）
• (2) 返回布尔型Series
• (3) df.loc[df['age'] > 30, 'salary'].mean()

解析

54 • 填空 / 简答

阅读以下代码，回答问题：

```
from pathlib import Path
p = Path('/home/user/documents/report.pdf')
print(p.parent)
print(p.name)
print(p.suffix)
print(p.stem)
```

问题：

- (1) 四个 print 语句分别输出什么？
- (2) 如何获取该文件的绝对路径？
- (3) 如何检查该文件是否存在？

答案 对象操作

- (1)
  ```
  /home/user/documents
  report.pdf
  .pdf
  report
  ```
- (2) 使用 p.absolute() 或 p.resolve()
- (3) 使用 p.exists()

解析

55 • 填空 / 简答

阅读以下代码，回答问题：

```
def apply_operations(*funcs):
    def wrapper(x):
        result = x
        for func in funcs:
            result = func(result)
        return result
    return wrapper
double = lambda x: x * 2
add_ten = lambda x: x + 10
operation = apply_operations(double, add_ten)
print(operation(5))
```

问题：

- (1) 输出结果是什么？
- (2) 这段代码体现了什么设计模式？
- (3) *funcs 的作用是什么？

答案 • (1) 输出：20（5*2=10，10+10=20）

- (2) 装饰器模式或函数组合模式
- (3) *funcs 接收任意数量的函数作为参数

解析

五、程序设计题

56 • 填空 / 简答

设计一个 Student 类，要求：

- 属性：姓名、学号、成绩字典（科目：分数）
- 方法：
  add_score(subject, score) ：添加成绩
  get_average() ：计算平均分
  __str__() ：返回学生信息
  使用 @property 实现 total_score 属性
- 创建3个学生实例，使用列表推导式找出平均分大于80的学生

答案

```
class Student:
    def __init__(self, name, student_id):
        self.name = name
        self.student_id = student_id
        self.scores = {}
    def add_score(self, subject, score):
        self.scores[subject] = score
    def get_average(self):
        if not self.scores:
            return 0
        return sum(self.scores.values()) / len(self.scores)
    @property
    def total_score(self):
        return sum(self.scores.values())
    def __str__(self):
        return f"学生：{self.name}，学号：{self.student_id}，平均分：{self.get_average():.2f}"
s1 = Student("张三", "001")
s1.add_score("语文", 85)
s1.add_score("数学", 90)
s1.add_score("英语", 88)
s2 = Student("李四", "002")
s2.add_score("语文", 75)
s2.add_score("数学", 80)
s2.add_score("英语", 78)
s3 = Student("王五", "003")
s3.add_score("语文", 92)
s3.add_score("数学", 95)
s3.add_score("英语", 90)
students = [s1, s2, s3]
high_achievers = [s for s in students if s.get_average() > 80]
print("平均分大于80的学生：")
for student in high_achievers:
    print(student)
```

解析

57 • 填空 / 简答

使用 pathlib 编写一个函数 process_directory(path) ，要求：

- 接收一个目录路径

- 使用生成器返回该目录下所有 .txt 文件的完整路径
- 统计文件总数
- 打印每个文件的名称和大小

答案
```
from pathlib import Path
def process_directory(path):
    directory = Path(path)
    if not directory.exists() or not directory.is_dir():
        print("目录不存在")
        return
    txt_files = list(directory.glob('*.txt'))
    print(f"找到{len(txt_files)} 个txt文件")
    for file in txt_files:
        yield file
def file_generator(path):
    for file_path in process_directory(path):
        size = file_path.stat().st_size
        print(f"文件：{file_path.name}，大小：{size} 字节")
        yield file_path
# 使用示例
# for file in file_generator('/path/to/directory'):
# print(file)
```

解析

58 • 填空 / 简答

使用Pandas完成以下任务：

- 创建一个包含学生信息的DataFrame（姓名、语文、数学、英语）
- 计算每个学生的总分和平均分
- 找出总分最高的学生
- 使用 groupby 按平均分段（优秀>=90，良好>=80，及格>=60）统计人数
- 将结果保存到CSV文件

答案
```
import pandas as pd
df = pd.DataFrame({
    '姓名': ['张三', '李四', '王五', '赵六', '钱七'],
    '语文': [85, 92, 78, 88, 95],
    '数学': [90, 88, 85, 92, 87],
    '英语': [88, 95, 82, 90, 93]
})
df['总分'] = df[['语文', '数学', '英语']].sum(axis=1)
df['平均分'] = df[['语文', '数学', '英语']].mean(axis=1)
top_student = df.loc[df['总分'].idxmax()]
print(f"总分最高的学生：{top_student['姓名']}，总分：{top_student['总分']}")
def grade_level(avg):
    if avg >= 90:
        return '优秀'
    elif avg >= 80:
        return '良好'
    elif avg >= 60:
```

```
        return '及格'
    else:
        return '不及格'
df['等级'] = df['平均分'].apply(grade_level)
grade_stats = df.groupby('等级').size()
print("\n各等级人数统计：")
print(grade_stats)
df.to_csv('student_scores.csv', index=False, encoding='utf-8-sig')
print("\n数据已保存到 student_scores.csv")
```

解析

59 • 填空 / 简答

编写一个装饰器 timer ，要求：

- 测量函数执行时间
- 打印函数名和执行时间
- 使用该装饰器装饰一个计算斐波那契数列的函数
- 对比递归和迭代两种实现的性能

答案

```
import time
def timer(func):
    def wrapper(*args, **kwargs):
        start_time = time.time()
        result = func(*args, **kwargs)
        end_time = time.time()
        print(f"函数{func.__name__} 执行时间：{end_time - start_time:.6f} 秒")
        return result
    return wrapper
@timer
def fibonacci_recursive(n):
    if n <= 1:
        return n
    return fibonacci_recursive(n-1) + fibonacci_recursive(n-2)
@timer
def fibonacci_iterative(n):
    if n <= 1:
        return n
    a, b = 0, 1
    for _ in range(2, n+1):
        a, b = b, a + b
    return b
print("递归实现：")
print(f"结果：{fibonacci_recursive(30)}")
print("\n迭代实现：")
print(f"结果：{fibonacci_iterative(30)}")
```

解析

60 • 填空 / 简答

实现一个自定义迭代器类 EvenNumbers ，要求：

- 接收起始值和结束值
- 迭代返回范围内的所有偶数
- 实现 __iter__() 和 __next__() 方法
- 支持 for 循环遍历
- 演示使用该迭代器

答案

```
class EvenNumbers:
    def __init__(self, start, end):
        self.start = start if start % 2 == 0 else start + 1
        self.end = end
        self.current = self.start
    def __iter__(self):
        return self
    def __next__(self):
        if self.current > self.end:
            raise StopIteration
        result = self.current
        self.current += 2
        return result
even_nums = EvenNumbers(1, 10)
print("1到10之间的偶数：")
for num in even_nums:
    print(num, end=' ')
print()
even_nums2 = EvenNumbers(0, 20)
print("\n0到20之间的偶数：")
print(list(even_nums2))
程序设计题详细参考答案
题目1：学生成绩管理系统（完整实现）
"""
学生成绩管理系统
功能：添加学生、查询成绩、计算统计信息、生成报告
"""
class Student:
    """学生类 - 存储单个学生的信息"""
    def __init__(self, student_id, name, scores):
        """
        初始化学生对象
        :param student_id: 学号
        :param name: 姓名
        :param scores: 成绩字典，如 {'语文': 90, '数学': 85}
        """
        self.student_id = student_id
        self.name = name
        self.scores = scores
    def average_score(self):
        """计算平均分"""
```

```
        if not self.scores:
            return 0
        return sum(self.scores.values()) / len(self.scores)
    def total_score(self):
        """计算总分"""
        return sum(self.scores.values())
    def __str__(self):
        """字符串表示"""
        avg = self.average_score()
        return f"学号: {self.student_id}, 姓名: {self.name}, 平均分: {avg:.2f}"
    def __repr__(self):
        return f"Student({self.student_id}, {self.name}, {self.scores})"
class GradeManager:
"""成绩管理类 - 管理所有学生"""
def __init__(self):
    """初始化管理器"""
    self.students = {}
# 使用字典存储，key为学号
def add_student(self, student):
    """
    添加学生
    :param student: Student对象
    """
    if student.student_id in self.students:
        print(f"警告: 学号{student.student_id} 已存在，将覆盖原数据")
    self.students[student.student_id] = student
    print(f"成功添加学生: {student.name}")
def find_student(self, student_id):
    """
    查找学生
    :param student_id: 学号
    :return: Student对象或None
    """
    return self.students.get(student_id)
def get_subject_average(self, subject):
    """
    计算某科目的平均分
    :param subject: 科目名称
    :return: 平均分
    """
    scores = [s.scores.get(subject, 0) for s in self.students.values()
            if subject in s.scores]
    if not scores:
        return 0
    return sum(scores) / len(scores)
def get_top_students(self, n=3):
    """
    获取成绩前N名的学生
```

```
    :param n: 返回前几名
        :return: 学生列表
        """
        sorted_students = sorted(self.students.values(),
                key=lambda s: s.average_score(),
                reverse=True)
        return sorted_students[:n]
    def generate_report(self):
        """生成成绩报告"""
        print("\n" + "="*50)
        print("成绩统计报告".center(50))
        print("="*50)
        print(f"\n总学生数: {len(self.students)}")
        # 各科平均分
        if self.students:
            all_subjects = set()
            for student in self.students.values():
                all_subjects.update(student.scores.keys())
            print("\n各科平均分:")
            for subject in sorted(all_subjects):
                avg = self.get_subject_average(subject)
                print(f"
                {subject}: {avg:.2f}")
        # 前三名学生
        print("\n成绩前三名:")
        for i, student in enumerate(self.get_top_students(3), 1):
            print(f"
                {i}. {student}")
        print("="*50 + "\n")
# 使用示例
if __name__ == "__main__":
    # 创建管理器
    manager = GradeManager()
    # 添加学生
s1 = Student("001", "张三", {"语文": 90, "数学": 85, "英语": 88})
s2 = Student("002", "李四", {"语文": 85, "数学": 92, "英语": 90})
s3 = Student("003", "王五", {"语文": 88, "数学": 87, "英语": 85})
s4 = Student("004", "赵六", {"语文": 92, "数学": 88, "英语": 91})
manager.add_student(s1)
manager.add_student(s2)
manager.add_student(s3)
manager.add_student(s4)
# 查询学生
student = manager.find_student("001")
if student:
    print(f"\n查询结果: {student}")
# 生成报告
manager.generate_report()
```

设计要点：
1. 面向对象设计
面向对象设计：Student类负责单个学生数据，GradeManager负责整体管理
2. 数据结构选择
数据结构选择：使用字典存储学生，便于快速查找
3. 错误处理
错误处理：检查学号是否重复，处理空数据情况
4. 代码复用
代码复用：average_score等方法可被多处调用
5. 可扩展性
可扩展性：易于添加新功能（如删除学生、修改成绩等）
题目2：装饰器实现（完整实现）
"""
装饰器实现：日志记录、性能测试、缓存
"""

```
import time
import functools
from datetime import datetime
def log_function_call(func):
    """
    日志装饰器 - 记录函数调用信息
    功能：记录函数名、参数、返回值、调用时间
    """
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        # 记录调用时间
        call_time = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
        # 记录参数
        args_str = ', '.join(repr(arg) for arg in args)
        kwargs_str = ', '.join(f"{k}={v!r}" for k, v in kwargs.items())
        all_args = ', '.join(filter(None, [args_str, kwargs_str]))
        print(f"[{call_time}] 调用函数: {func.__name__}({all_args})")
        # 执行函数
        try:
            result = func(*args, **kwargs)
            print(f"[{call_time}] 返回值: {result!r}")
            return result
        except Exception as e:
            print(f"[{call_time}] 异常: {type(e).__name__}: {e}")
            raise
    return wrapper
def timer(func):
    """
    性能测试装饰器 - 测量函数执行时间
    """
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        start_time = time.time()
```

```
        result = func(*args, **kwargs)
        end_time = time.time()
        elapsed_time = end_time - start_time
        print(f"函数{func.__name__} 执行时间: {elapsed_time:.6f} 秒")
        return result
    return wrapper
def cache(func):
    """
    缓存装饰器 - 缓存函数结果
    使用字典存储已计算的结果，避免重复计算
    """
    cached_results = {}
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        # 创建缓存键（注意：kwargs需要转换为可哈希类型）
        cache_key = (args, tuple(sorted(kwargs.items())))
        if cache_key in cached_results:
            print(f"缓存命中: {func.__name__}{args}")
            return cached_results[cache_key]
        # 计算结果并缓存
        result = func(*args, **kwargs)
        cached_results[cache_key] = result
        print(f"缓存存储: {func.__name__}{args} = {result}")
        return result
    # 添加缓存管理方法
    wrapper.cache_info = lambda: {
        'size': len(cached_results),
        'hits': sum(1 for _ in cached_results)
    }
    wrapper.cache_clear = lambda: cached_results.clear()
    return wrapper
# 组合使用多个装饰器
@log_function_call
@timer
@cache
def fibonacci(n):
    """计算斐波那契数列第n项"""
    if n <= 1:
        return n
    return fibonacci(n-1) + fibonacci(n-2)
@log_function_call
def add(a, b):
    """加法函数"""
    return a + b
# 使用示例
if __name__ == "__main__":
    print("=" * 60)
    print("测试日志装饰器")
```

```
    print("=" * 60)
    result = add(3, 5)
    print("\n" + "=" * 60)
    print("测试缓存装饰器")
    print("=" * 60)
    fibonacci(5)
                # 第二次调用会使用缓存
print("\n" + "=" * 60)
print("测试性能装饰器")
print("=" * 60)
@timer
def slow_function():
    time.sleep(0.1)
    return "完成"
slow_function()
设计要点：
1. functools.wraps：保留原函数的元数据（名称、文档等）
2. 闭包
闭包：装饰器内部函数可以访问外部变量（如cached_results）
3. 可组合性
可组合性：多个装饰器可以叠加使用
4. 缓存键设计
缓存键设计：将args和kwargs转换为可哈希的元组
5. 扩展性
扩展性：为cache装饰器添加cache_info和cache_clear方法
题目3：生成器实现（完整实现）
"""
生成器实现：文件读取、数据过滤、无限序列
"""
from pathlib import Path
import itertools
def read_large_file(filepath, chunk_size=1024):
    """
    生成器：逐块读取大文件
    :param filepath: 文件路径
    :param chunk_size: 每次读取的字节数
    :yield: 文件内容块
    """
    try:
        with open(filepath, 'r', encoding='utf-8') as f:
            while True:
                chunk = f.read(chunk_size)
                if not chunk:
                break
                yield chunk
    except FileNotFoundError:
        print(f"文件不存在: {filepath}")
    except Exception as e:
```

```
        print(f"读取文件出错: {e}")
def read_lines(filepath):
    """
    生成器：逐行读取文件
    :param filepath: 文件路径
    :yield: 文件的每一行
    """
    try:
        with open(filepath, 'r', encoding='utf-8') as f:
            for line in f:
                yield line.rstrip('\n')
    except Exception as e:
        print(f"读取文件出错: {e}")
def filter_data(data_iter, condition):
    """
    生成器：过滤数据
    :param data_iter: 数据迭代器
    :param condition: 过滤条件函数
    :yield: 满足条件的数据
    """
    for item in data_iter:
        if condition(item):
            yield item
def fibonacci_generator():
    """
    生成器：无限斐波那契数列
    :yield: 斐波那契数
    """
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a + b
def prime_generator():
    """
    生成器：无限素数序列
    :yield: 素数
    """
    def is_prime(n):
        if n < 2:
            return False
        for i in range(2, int(n ** 0.5) + 1):
            if n % i == 0:
                return False
        return True
    n = 2
    while True:
        if is_prime(n):
            yield n
```

```
        n += 1
def batch_generator(iterable, batch_size):
    """
    生成器：将数据分批
    :param iterable: 可迭代对象
    :param batch_size: 批次大小
    :yield: 数据批次
    """
    iterator = iter(iterable)
    while True:
        batch = list(itertools.islice(iterator, batch_size))
        if not batch:
            break
        yield batch
# 使用示例
if __name__ == "__main__":
    print("=" * 60)
    print("示例1: 斐波那契数列生成器")
    print("=" * 60)
    fib = fibonacci_generator()
    print("前10个斐波那契数:")
    for i, num in enumerate(fib):
        if i >= 10:
            break
        print(num, end=' ')
    print("\n")
    print("=" * 60)
    print("示例2: 素数生成器")
    print("=" * 60)
    primes = prime_generator()
    print("前10个素数:")
    for i, prime in enumerate(primes):
    if i >= 10:
        break
    print(prime, end=' ')
print("\n")
print("=" * 60)
print("示例3: 数据过滤")
print("=" * 60)
numbers = range(1, 21)
even_numbers = filter_data(numbers, lambda x: x % 2 == 0)
print("1-20的偶数:")
print(list(even_numbers))
print("\n" + "=" * 60)
print("示例4: 批次生成器")
print("=" * 60)
data = range(1, 11)
batches = batch_generator(data, 3)
```

```
print("将1-10分成批次(每批3个):")
for i, batch in enumerate(batches, 1):
    print(f"批次{i}: {batch}")
# 文件操作示例（需要实际文件）
print("\n" + "=" * 60)
print("示例5: 文件读取生成器")
print("=" * 60)
# 创建测试文件
test_file = Path("test_data.txt")
test_file.write_text("第一行\n第二行\n第三行\n第四行\n第五行\n")
print("逐行读取文件:")
for line in read_lines(test_file):
    print(f"
            {line}")
# 清理测试文件
test_file.unlink()
设计要点：
1. 内存效率
内存效率：生成器逐个产生数据，不需要一次性加载所有数据
2. 惰性计算
惰性计算：只在需要时才计算下一个值
3. 无限序列
无限序列：可以生成无限序列（如斐波那契、素数）
4. 组合使用
组合使用：生成器可以串联使用（如filter_data）
5. 资源管理
资源管理：使用with语句确保文件正确关闭
题目4：Pandas数据分析（完整实现）
"""
Pandas数据分析：数据清洗、统计分析、可视化
"""
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from datetime import datetime, timedelta
# 设置中文显示
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False
class DataAnalyzer:
    """数据分析类"""
    def __init__(self, data=None):
        """初始化分析器"""
        self.df = data if data is not None else pd.DataFrame()
    def load_data(self, filepath):
        """加载数据"""
        try:
            self.df = pd.read_csv(filepath)
            print(f"成功加载数据: {len(self.df)} 行")
```

```
            return self.df
        except Exception as e:
            print(f"加载数据失败: {e}")
            return None
    def clean_data(self):
        """数据清洗"""
        print("\n开始数据清洗...")
        original_size = len(self.df)
        # 1. 删除完全重复的行
        self.df.drop_duplicates(inplace=True)
        print(f"删除重复行: {original_size - len(self.df)} 行")
    # 2. 处理缺失值
    missing_before = self.df.isnull().sum().sum()
    # 数值列用中位数填充
    numeric_cols = self.df.select_dtypes(include=[np.number]).columns
    for col in numeric_cols:
        self.df[col].fillna(self.df[col].median(), inplace=True)
    # 分类列用众数填充
    categorical_cols = self.df.select_dtypes(include=['object']).columns
    for col in categorical_cols:
        mode_value = self.df[col].mode()
        if len(mode_value) > 0:
            self.df[col].fillna(mode_value[0], inplace=True)
    missing_after = self.df.isnull().sum().sum()
    print(f"处理缺失值: {missing_before - missing_after} 个")
    # 3. 删除异常值（使用IQR方法）
    for col in numeric_cols:
        Q1 = self.df[col].quantile(0.25)
        Q3 = self.df[col].quantile(0.75)
        IQR = Q3 - Q1
        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR
        outliers = ((self.df[col] < lower_bound) |
                (self.df[col] > upper_bound)).sum()
        if outliers > 0:
            print(f"列{col} 发现{outliers} 个异常值")
            self.df = self.df[
                (self.df[col] >= lower_bound) &
                (self.df[col] <= upper_bound)
            ]
    print(f"清洗完成，剩余{len(self.df)} 行\n")
    return self.df
def basic_statistics(self):
    """基础统计分析"""
    print("=" * 60)
    print("基础统计信息")
    print("=" * 60)
    print(self.df.describe())
```

```
    print("\n数据类型:")
    print(self.df.dtypes)
    print(f"\n数据形状: {self.df.shape}")
    return self.df.describe()
def group_analysis(self, group_by, agg_col, agg_func='mean'):
    """分组分析"""
    print(f"\n按{group_by} 分组，对{agg_col} 进行{agg_func} 统计:")
    result = self.df.groupby(group_by)[agg_col].agg(agg_func)
    print(result)
    return result
def visualize(self, save_path='analysis_report.png'):
    """数据可视化"""
    numeric_cols = self.df.select_dtypes(include=[np.number]).columns
    if len(numeric_cols) == 0:
        print("没有数值列可以可视化")
        return
    n_cols = min(len(numeric_cols), 4)
    fig, axes = plt.subplots(2, 2, figsize=(12, 10))
    axes = axes.flatten()
    for i, col in enumerate(numeric_cols[:4]):
        # 直方图
        axes[i].hist(self.df[col].dropna(), bins=30, edgecolor='black')
        axes[i].set_title(f'{col} 分布')
        axes[i].set_xlabel(col)
        axes[i].set_ylabel('频数')
    plt.tight_layout()
    plt.savefig(save_path)
    print(f"\n可视化图表已保存: {save_path}")
    plt.close()
# 使用示例
if __name__ == "__main__":
    # 创建示例数据
    np.random.seed(42)
    dates = pd.date_range('2024-01-01', periods=100, freq='D')
    df = pd.DataFrame({
        '日期': dates,
        '销售额': np.random.normal(10000, 2000, 100),
        '访客数': np.random.randint(100, 500, 100),
        '转化率': np.random.uniform(0.01, 0.1, 100),
        '地区': np.random.choice(['北京', '上海', '广州', '深圳'], 100)
    })
    # 添加一些缺失值和异常值
    df.loc[np.random.choice(df.index, 5), '销售额'] = np.nan
    df.loc[np.random.choice(df.index, 3), '销售额'] = 50000
    # 异常值
    # 创建分析器
    analyzer = DataAnalyzer(df)
    # 数据清洗
```

```
    analyzer.clean_data()
    # 基础统计
    analyzer.basic_statistics()
    # 分组分析
    analyzer.group_analysis('地区', '销售额', 'mean')
    analyzer.group_analysis('地区', '访客数', 'sum')
    # 可视化
    analyzer.visualize()
    print("\n分析完成！")
```

设计要点：

1. 数据清洗流程

数据清洗流程：去重→处理缺失值→删除异常值

2. 统计方法

统计方法：使用IQR方法检测异常值

3. 分组聚合

分组聚合：灵活的groupby操作

4. 可视化

可视化：使用matplotlib生成图表

5. 面向对象

面向对象：封装成类便于复用

题目5：Pathlib文件操作（完整实现）

```
"""
Pathlib文件操作：文件管理、目录遍历、批量处理
"""
from pathlib import Path
import shutil
from datetime import datetime
class FileManager:
    """文件管理器类"""
    def __init__(self, base_path='.'):
        """初始化文件管理器"""
        self.base_path = Path(base_path)
        if not self.base_path.exists():
            self.base_path.mkdir(parents=True)
    def create_directory_structure(self, structure):
        """
        创建目录结构
        :param structure: 目录结构字典
        """
        for dir_name in structure:
            dir_path = self.base_path / dir_name
            dir_path.mkdir(parents=True, exist_ok=True)
            print(f"创建目录: {dir_path}")
    def find_files(self, pattern='*', recursive=True):
        """
        查找文件
        :param pattern: 文件模式（如 '*.txt'）
        :param recursive: 是否递归查找
```

```
        :return: 文件路径列表
        """
        if recursive:
            files = list(self.base_path.rglob(pattern))
        else:
            files = list(self.base_path.glob(pattern))
    print(f"找到{len(files)} 个匹配的文件")
    return files
def organize_by_extension(self, source_dir=None):
    """
    按扩展名组织文件
    将文件移动到以扩展名命名的子目录中
    """
    source = Path(source_dir) if source_dir else self.base_path
    for file_path in source.iterdir():
        if file_path.is_file():
            # 获取扩展名（去掉点号）
            ext = file_path.suffix[1:] if file_path.suffix else 'no_extension'
            # 创建目标目录
            target_dir = source / ext
            target_dir.mkdir(exist_ok=True)
            # 移动文件
            target_path = target_dir / file_path.name
            shutil.move(str(file_path), str(target_path))
            print(f"移动: {file_path.name} -> {ext}/")
def batch_rename(self, pattern, prefix='', suffix=''):
    """
    批量重命名文件
    :param pattern: 文件模式
    :param prefix: 前缀
    :param suffix: 后缀
    """
    files = self.find_files(pattern, recursive=False)
    for i, file_path in enumerate(files, 1):
        new_name = f"{prefix}{i:03d}{suffix}{file_path.suffix}"
        new_path = file_path.parent / new_name
        file_path.rename(new_path)
        print(f"重命名: {file_path.name} -> {new_name}")
def get_file_info(self, filepath):
    """
    获取文件信息
    :param filepath: 文件路径
    :return: 文件信息字典
    """
    path = Path(filepath)
    if not path.exists():
        return None
    stat = path.stat()
```

```
    info = {
        '文件名': path.name,
        '完整路径': str(path.absolute()),
        '大小(字节)': stat.st_size,
        '创建时间': datetime.fromtimestamp(stat.st_ctime),
        '修改时间': datetime.fromtimestamp(stat.st_mtime),
        '是否文件': path.is_file(),
        '是否目录': path.is_dir(),
        '扩展名': path.suffix
    }
    return info
def search_content(self, keyword, file_pattern='*.txt'):
    """
    在文件中搜索关键词
    :param keyword: 搜索关键词
    :param file_pattern: 文件模式
    :return: 包含关键词的文件列表
    """
    results = []
    files = self.find_files(file_pattern)
    for file_path in files:
        try:
            content = file_path.read_text(encoding='utf-8')
            if keyword in content:
                results.append(file_path)
                print(f"找到: {file_path}")
            except Exception as e:
                print(f"读取文件失败{file_path}: {e}")
        return results
    def backup_files(self, backup_dir='backup'):
        """
        备份文件
        :param backup_dir: 备份目录名
        """
        backup_path = self.base_path / backup_dir
        backup_path.mkdir(exist_ok=True)
        timestamp = datetime.now().strftime('%Y%m%d_%H%M%S')
        for file_path in self.base_path.iterdir():
            if file_path.is_file():
                backup_name = f"{file_path.stem}_{timestamp}{file_path.suffix}"
                backup_file = backup_path / backup_name
                shutil.copy2(str(file_path), str(backup_file))
                print(f"备份: {file_path.name} -> {backup_name}")
# 使用示例
if __name__ == "__main__":
    # 创建测试环境
    test_dir = Path("test_file_manager")
    test_dir.mkdir(exist_ok=True)
```

```
    # 创建文件管理器
    fm = FileManager(test_dir)
    print("=" * 60)
    print("示例1: 创建目录结构")
    print("=" * 60)
    structure = ['documents', 'images', 'videos', 'backup']
    fm.create_directory_structure(structure)
    print("\n" + "=" * 60)
print("示例2: 创建测试文件")
print("=" * 60)
# 创建一些测试文件
(test_dir / 'file1.txt').write_text('这是测试文件1')
(test_dir / 'file2.txt').write_text('这是测试文件2，包含关键词Python')
(test_dir / 'file3.md').write_text('# Markdown文件')
(test_dir / 'image.jpg').touch()
print("\n" + "=" * 60)
print("示例3: 查找文件")
print("=" * 60)
txt_files = fm.find_files('*.txt', recursive=False)
print("\n" + "=" * 60)
print("示例4: 获取文件信息")
print("=" * 60)
if txt_files:
    info = fm.get_file_info(txt_files[0])
    for key, value in info.items():
        print(f"{key}: {value}")
print("\n" + "=" * 60)
print("示例5: 搜索内容")
print("=" * 60)
fm.search_content('Python', '*.txt')
print("\n" + "=" * 60)
print("示例6: 备份文件")
print("=" * 60)
fm.backup_files()
# 清理测试环境
print("\n清理测试环境...")
shutil.rmtree(test_dir)
print("完成！")
```

设计要点：

1. Path对象

    对象：使用pathlib.Path代替字符串路径操作

2. 跨平台

跨平台：Path自动处理不同操作系统的路径分隔符

3. 链式操作

链式操作：Path支持/操作符连接路径

4. 异常处理

异常处理：处理文件不存在、权限等问题

5. 实用功能

实用功能：文件组织、批量重命名、内容搜索、备份等
评分说明：
- 单选题：每题1分，共30分
- 多选题：每题2分，选对但不全得1分，有错选不得分，共30分
- 填空题：每空1分，共10分
- 代码阅读题：每题3分，共15分
- 程序设计题：5选3，每题5分，共15分
- 总分：

总分：100分

解析